

FS Shooter System

Create dynamic, tactical third-person shooters in Unity with smart AI, built-in weapons, and modular integration. This system provides everything you need to build responsive ranged combat, from cover-based firefights to authored enemy encounters. Whether you're making a full-scale third-person shooter or an FPS, this system gives you the tools to bring it to life.

Key Features:

- 6 Built-in Weapons - Pistol, Rifle, Sniper, Bazooka, Arrow, and Grenade — ready to use out of the box. Easily create custom weapons with your own stats without writing any code.
- Tactical Cover System - Both the player and AI can take cover behind obstacles, peek out, and fire tactically.
- Advanced Enemy AI - Enemies make strategic decisions — searching for optimal cover based on line of sight, distance, and angle of attack.
- Flanking System - If the player stays in cover for too long, AI units will flank from alternate positions, forcing players to remain mobile.
- Authored Enemy Behaviors - Design cinematic or challenging encounters by assigning specific behaviors and actions to AI in different zones or moments.
- Modular Integration with other FS Systems - Seamlessly works with other FS Systems like Melee Combat System, Parkour and Climbing System, Inventory System, etc.

Quick Start

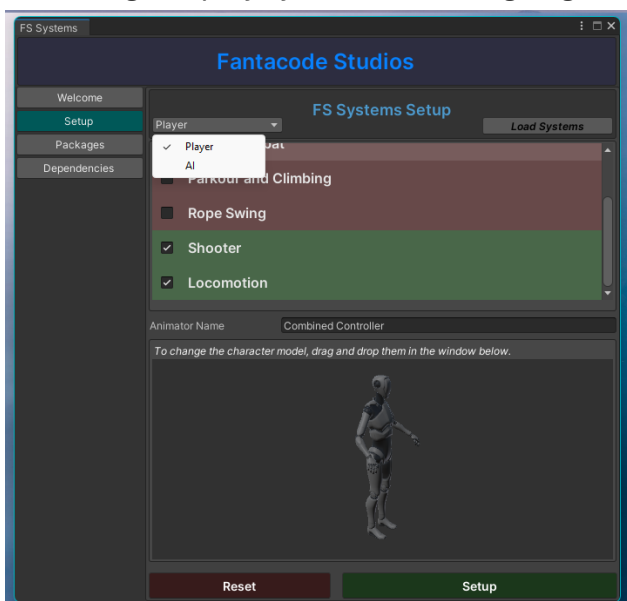


✓ 1. Create Characters

1. Open the FS Systems Setup Window

Open the **FS Systems** window from the Unity editor by navigating to: **Tools > FS Systems**

This window allows you to quickly configure and initialize both **player** and **AI** characters by selecting relevant gameplay systems and assigning a character model.



2. Select Character Type

At the top of the window, use the dropdown menu to choose the character type:

- **Player:** For characters controlled by the player.
- **AI:** For non-player characters (NPCs), such as enemies or allies.

✓ Select **Player** to create a controllable character.

3. Choose Gameplay Systems

In the **Systems List**, enable the modules you want to include by checking their boxes:
Each module adds specific gameplay functionality to your character.

4. Assign a Character Model

Drag and drop your 3D character model (e.g., a **humanoid prefab**) from the **Project window** into the preview area at the bottom of the setup window.

This will link your character model with the configured systems.

⚠ Ensure your model has a valid **Humanoid Avatar** assigned in its import settings for correct animation functionality.

Step 5: Click "Setup"

Once you've configured the systems and model, click the **Setup** button to automatically add and configure all required components (scripts, animator controller, etc.).

Step 6: Create NPCs (Enemies & AI Characters)

If your system provides **AI support** for systems like **Shooter**, **Melee**, etc..

After configuring your **player character**, the next step is to design and set up **NPCs (Non-Player Characters)**, such as enemies, allies, based on your game's mechanics and intended experience.



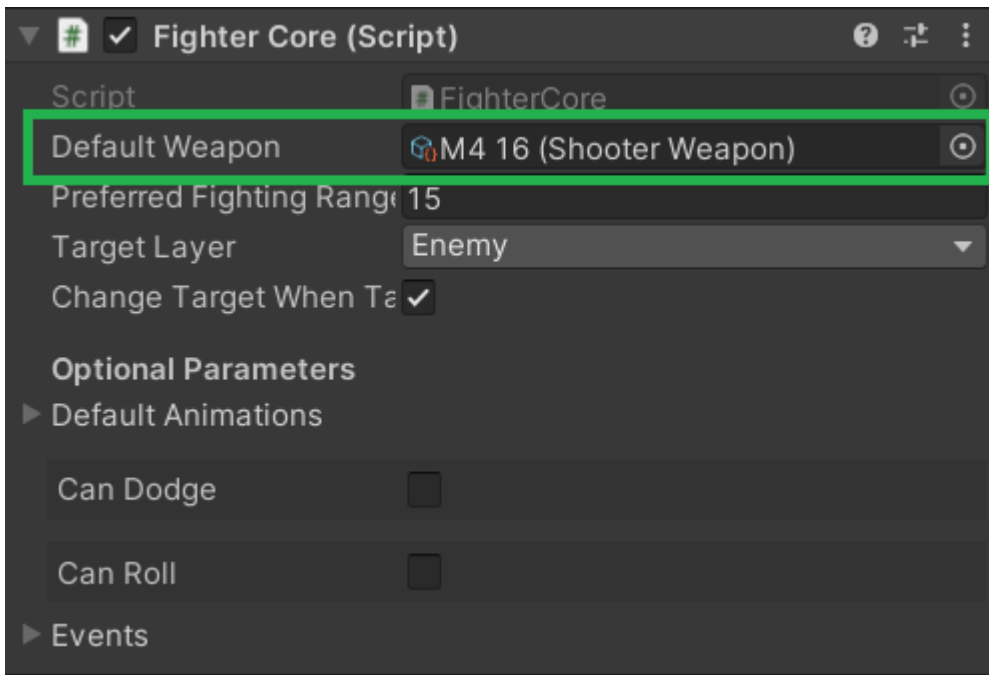
- Open the **FS Systems Setup** window.
- Change the **Character Type** dropdown to **AI**.
- Select the appropriate systems (e.g., Shooter, Melee).
- Drag and drop the desired character model.
- Click **Setup** to auto-configure components for AI behavior.

2. Default Weapon Setup

Setting up a **default weapon** ensures that a character (Player or AI) automatically starts the game with a weapon already equipped whether it's a melee weapon, shooter weapon, or any other type supported by the system.

Where to Set It

You can configure the **Default Weapon** in the **Fighter Core** component. Both Player and AI characters that use this script



- Select the GameObject that contains the Fighter Core script (Player or AI character)
- In the **Inspector**, locate the Default Weapon field on the Fighter Core component.
- **Assign the Weapon ScriptableObject** to this field by dragging it from the Project window.

3. AI Navigation: Setting Up NavMesh Surface

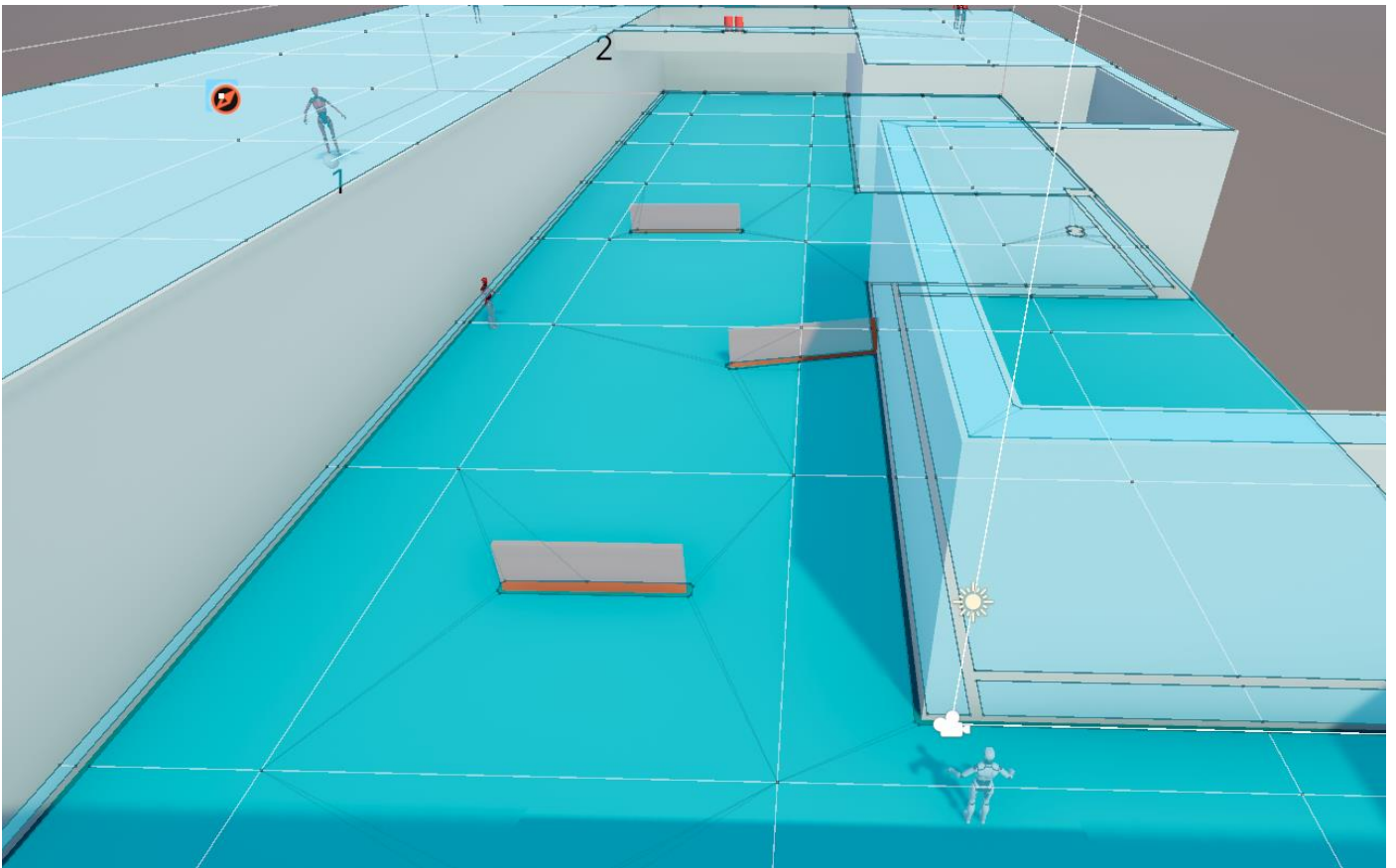
To allow AI characters to move and navigate your game environment, you must set up Unity's **NavMesh Surface**. This enables the AI system to understand the walkable areas of your scene.

Add a NavMeshSurface to the Scene

- In the **Hierarchy**, create or select a GameObject to hold the navigation surface (commonly an empty GameObject named Navigation or NavMesh).
- In the **Inspector**, click **Add Component** and search for NavMeshSurface (from Unity's AI Navigation package).

Bake the NavMesh

- Click the **Bake** button in the NavMeshSurface component.
- Unity will scan the scene and generate a navigation mesh where AI can move.

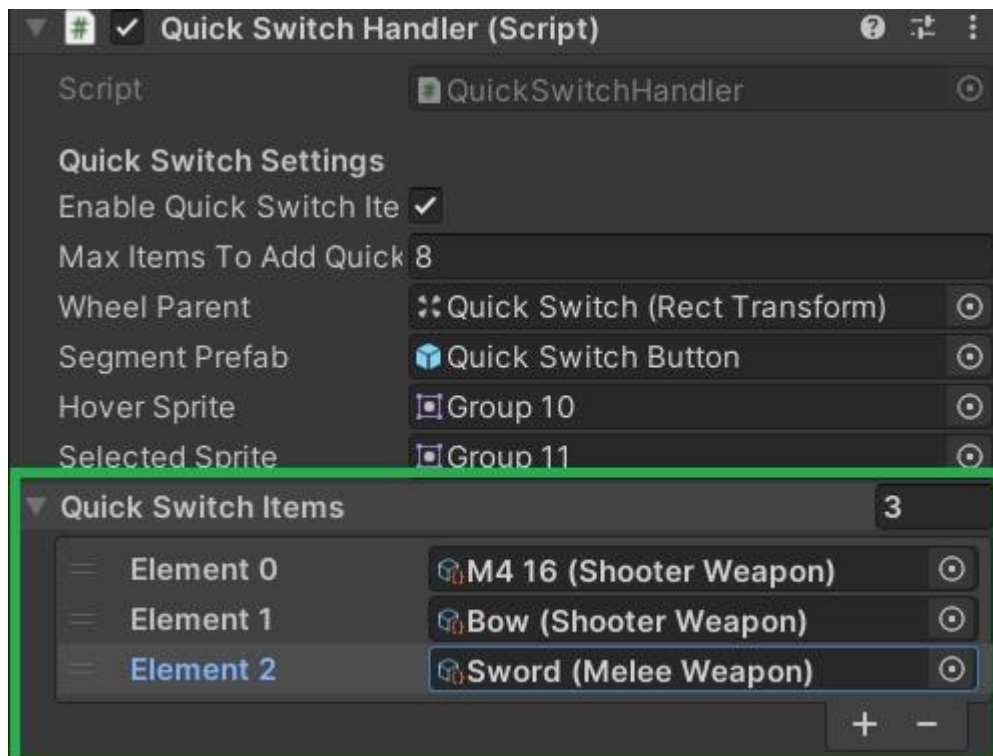


4. Weapon Wheel System

The **Weapon Wheel** provides a fast and intuitive way for players to switch between weapons and tools during gameplay.



The QuickSwitchHandler is a script attached to the Player GameObject that enables fast switching between weapons. You can manually add default items to the weapon wheel before starting to play, and any equipped weapon will automatically be added to the wheel during gameplay.



If your system supports weapons such as:

- 🗡️ **Melee Weapons** (fists, swords, knives)
- 🏹 **Shooter Weapons** (guns, crossbows)
- li>• 💣 **Throwables** (grenades, bombs)
- 🪂 **Utility Tools** (grappling hooks)

...then the **Weapon Wheel** is a powerful interface to manage them.

- It allows the player to **quickly switch** between weapon types without navigating complex menus.
- You can dynamically populate the wheel based on the player's inventory or predefined loadout.

✅ This system streamlines gameplay and enhances combat flow, especially in fast-paced or mixed-ability games.

For more detailed documentation of the **Shooter System**, please check it out [here](#).